Ocarina of Time (Switch 2) on Mac — market fix + frame generation for nxnative

Two things for people running the Ocarina of Time Switch 2 remake through seedds/nxnative on a Mac:

1. **The market fix** — Hyrule Castle market runs at 10–16 fps on a base M5 with default settings. One line in `env.txt` fixes it to a locked 60.
2. **Frame generation** — a patch that displays 120 frames per second over the game's 60, by generating a frame between each pair of real ones.

Tested on: MacBook Pro, base M5 (16 GB), macOS 27.2, Ocarina of Time Switch 2 remake (title `0400486024DEC000`, update v131072). Nothing here is upstream or official. You need your own game dump and a working nxnative build of the game first — this repo contains no game files.

---

1. The market fix (`DM_RT=0`)

The problem. On a base M5 the Hyrule Castle market ran at a sustained 10–16 fps while the rest of the game held 60. The frame-time log (slowframe receipts) showed the market frames were bound by the present thread's CPU time, not the GPU: frame times of 62–102 ms with modest GPU times and no new shader compiles.

The cause. The host declares a CPU-time budget for its present thread to macOS (the real-time time-constraint policy). The budget is auto-sized from the thread's recent frame times: it starts at 2 ms and re-sizes every 120 frames toward the measured peak, capped at 15 ms. On light scenes it learned about 4.6 ms on this machine. The market needs far more per frame than the learned budget, so macOS kept demoting the thread for overrunning its own declared budget — and the heavier the scene got, the worse the throttling bit. A feedback loop, not a lack of power: the same scene runs at 60 the moment the declaration is off.

The fix. Add one line to the game's `env.txt` (at `~/Library/Application Support/nxnative/0400486024DEC000/env.txt`):

```
DM_RT=0
```

That turns the declaration off entirely. No budget promised, no throttling. The market went from 10–16 fps to a flat 60, and a full session afterwards measured 0.44% slow frames across 11,390 judged frames.

The A/B that confirms it. The maintainer (lunks, on issue #1985) explained the auto-sizing and asked for a test with a fixed maximum declaration, `DM_RT_COMP=15`. Result: the market mostly held 60 with brief stutters, but the run from the market entrance to the main fountain locked at 55 fps, and once, idling at the market entrance right after loading, it fell to ~15 fps — neither happened with `DM_RT=0`. So the fixed big budget is better than the learned small one, and no declaration at all is better than both on a base M5.

2. Frame generation (`DM_FRAMEGEN`)

How it works. The port already presents the game on a 120 Hz display link (`DM_PRESENT_LINK=2`) while the game itself steps at 60 fps — so every other display tick has nothing new to show. The patch fills those ticks:

- Every real frame is copied into a small two-frame history as it is taken for presentation.
- On the tick a frame arrives, the screen shows instead a 50/50 blend of the previous frame and this one.
- The real frame itself is shown on the next tick.

That is 120 presented pictures per second over the same 60 real frames. The game still renders and simulates at exactly 60.

The accounting trick that makes it safe. Real frames land one tick (8.3 ms) later than stock. That delay is the host's own, so it is subtracted back out of the presented times the game uses for its frame pacing — the frame step judges intervals between presented times, and a constant offset cancels in the differences. Generated frames take no present ticket and write no presented-time entry. From the game's side, nothing changed.

When it runs. Generation engages only at a steady 60 fps cadence: the first engagement waits for ~10 seconds of steady frames (see the crash note below), later ones return after ~0.5 seconds. A tick of frame-time jitter is ridden through. An occasional slow frame is shown as-is without a blend and generation stays engaged; repeated slow frames, a loading stall, a scene change, or the GPU still being busy with the previous generated frame all drop it back to stock presents, and each quick drop-out makes it wait longer before returning.

Honest limits.

- This is stage 1: a plain colour blend, on purpose — it proves the scheduling and accounting. Fast camera spins ghost. A motion-compensated version (warping with motion data) would be stage 2; there is no motion or depth data available at the present point, which is also why MetalFX frame interpolation cannot be dropped in there.
- 120 shown is not 120 rendered. Input response stays at the game's 60 fps level, plus the one-tick delay.
- It does nothing for a scene that cannot hold 60 (it only engages at a steady 60). Fix the market first (above); the two stack.

Version history, briefly (kept because every failure taught the design something):

- **v1** engaged instantly and crashed seconds after boot, on the first input, with a SIGSEGV inside Apple's Metal shader-compiler cache (`fscache_find_and_retain`, on a shader-prefetch worker) while its own GPU work reported no errors. Suspected trigger: the blend's one pipeline compile landing in the boot window's compile storm. Hence the delayed first engagement.
- **v2** added the engagement gate, but the held-frame code consumed the tick counter the cadence check needed — generation blinked on alternate frames and the HUD read exactly 80 fps. Fixed by leaving the counters to the takes.
- **v3** fixed the gate being hair-triggered: ordinary jitter (a 17 ms frame landing 3 ticks later) dropped generation instantly and it never came back during play — 120 only while standing still. Fixed with jitter tolerance and a 0.5 s re-engagement.
- **v4** (this file) adds slow-frame forgiveness, a GPU/load watchdog with a doubling backoff, and a bounded 2 ms wait for the blend allocator (v1–v3 could in principle block the display-link thread indefinitely there).

Setup

env.txt

Full working `env.txt` (also in this repo as `env.txt.example`), at `~/Library/Application Support/nxnative/0400486024DEC000/env.txt`:

```
DM_PRESENT_LINK=2
DM_SLOWFRAME=1
DM_RT=0
DM_FRAMEGEN=1
```

- `DM_PRESENT_LINK=2` — the 120 Hz display link. Required for frame generation; test windowed (or add `DM_SCREEN_MATCH=0` for fullscreen, which otherwise pins the screen to a multiple of the game's rate).
- `DM_SLOWFRAME=1` — frame-time receipts in the log; diagnostic, optional.
- `DM_RT=0` — the market fix.
- `DM_FRAMEGEN=1` — frame generation on. Delete this line to turn it off instantly; stock behaviour, no rebuild.

Applying the patch and building

Against a current clone of seedds/nxnative (the patch applies to upstream HEAD as of 2026-11-10):

```
cd ~/nxnative-build
git checkout -b framegen-experiment
git apply framegen-stage1-v4.patch
make -C dm/host GAME=0400486024DEC000
python3 scripts/build_ipa_from_nsp.py "<folder with your game dump>" --platform mac-app
```

The `make` step is a compile check; it ends with warnings, not errors, when the patch is in correctly. The app build reuses your shader cache unless the translation chain changed upstream (a chain change means one full shader recompile, ~40 minutes for this game; ordinary rebuilds take minutes).

What you should see

- Log line at start: `frame generation on (DM_FRAMEGEN)`.
- After ~10 s of steady play: `frame generation engaged`, and the performance HUD steps from 60 to ~120 and *stays* there while you move.
- Slow-frame percentages stay in line with the no-generation reference (~0.4%), with no frame-step hold changes.
- Kill switch if anything misbehaves: remove the `DM_FRAMEGEN` line from `env.txt` and relaunch.

FAQ

So the 120 is fake? 120 displayed, 60 rendered. Half the pictures are generated blends — that is what frame generation means, DLSS included. Motion genuinely has twice the steps; responsiveness stays at the game's rate.

The market is still slow for me. Frame generation will not help there — it only engages at a steady 60. Do the `DM_RT=0` fix first.

Does this work on other games? The mechanism is general to any 60 fps game presented on the 120 Hz link, but only Ocarina of Time on a base M5 has been tested. Claims beyond that would be guessing.

Is this coming upstream? Not at the moment. It is a local experiment. If it ever goes further it needs the project's own process (a reference doc entry and regression checks); the issue thread is #1985 for the market half of the story.

Credits

- seedds/nxnative is the project that makes any of this possible; the market diagnosis was confirmed and sharpened by lunks on issue #1985.
- Found, measured, built and tested on a base M5 by Prashant (@PrashantJi02), iterating the patch design across four versions with AI assistance in drafting the code. Every build, test and

measurement in this repo was run on the real machine.